

Pengumpulan Tahap 2

Nama: Robani Diansyah

NPM: 23312246

PRODUCT-SERVICE

1. api/services/product-service/prisma/schema.prisma

```
// Buat model product
model Product {
  id          Int          @id @default(autoincrement())
  nama        String
  deskripsi   String?
  harga       Int
  stok        Int          @default(0)
  kategoriId  Int
  createdAt   DateTime     @default(now())
  updatedAt   DateTime     @updatedAt
}
```

File ini mendefinisikan struktur tabel Product di database. Prisma akan membaca file ini untuk generate tabel dan client.

Field	Tipe	Fungsi
Id	Int	Primary key, otomatis bertambah setiap ada data baru
Nama	String	Nama product, wajib diisi
deskripsi	String?	Deskripsi product, boleh kosong
Harga	Int	Harga product dalam rupiah, wajib diisi
Stok	Int	Jumlah stok tersedia, default 0 kalau tidak diisi
kategorid	Int	ID kategori yang dimiliki product ini
createdAt	DateTime	Waktu data dibuat, otomatis diisi Prisma
updateAt	DateTime	Waktu data terakhir diubah, otomatis diperbarui Prisma

File main.ts, app.module.ts, prisma.module.ts, dan prisma.service.ts sama persis dengan kategori-service hanya beda nama module dan port (product-service pakai port **3002**).

2. api/services/product-service/src/product/dto/create-product.dto.ts

```
import {
  IsString,
  IsOptional,
  IsInt,
  Min,
  MinLength,
  MaxLength,
} from 'class-validator';

// DTO untuk validasi request body saat create product
```

```
export class CreateProductDto {
  @IsString()
  @MinLength(3, { message: 'Nama minimal 3 karakter' })
  @MaxLength(100, { message: 'Nama maksimal 100 karakter' })
  nama!: string;

  @IsOptional()
  @IsString()
  @MaxLength(255, { message: 'Deskripsi maksimal 255 karakter' })
  deskripsi?: string;

  @IsInt({ message: 'Harga harus berupa angka' })
  @Min(0, { message: 'Harga tidak boleh negatif' })
  harga!: number;

  @IsInt({ message: 'Stok harus berupa angka' })
  @Min(0, { message: 'Stok tidak boleh negatif' })
  stok!: number;

  @IsInt({ message: 'Kategori ID harus berupa angka' })
  kategoriId!: number;
}
```

Sama seperti create-kategori.dto.ts tapi dengan tambahan field khusus product. Yang beda dari kategori adalah adanya validasi untuk harga, stok, dan kategorid.

Bagian	Fungsi
Nama	Sama seperti kategori wajib diisi, minimal 3 karakter, maksimal 100 karakter
Deskripsi	Sama seperti kategori opsional, maksimal 255 karakter
@IsInt()	Memastikan field harus berupa bilangan bulat, bukan desimal atau teks
@Min(0) pada harga	Harga tidak boleh negatif, minimal 0
@Min(0) pada stok	Stok tidak boleh negatif, minimal 0
kategorid	Wajib diisi dan harus berupa angka ini yang nantinya dipakai untuk relasi ke kategori-service

3. api/services/product-service/src/product/entities/product.entity.ts

```
// Model Product sesuai schema Prisma
export class Product {
  id!: number;
  nama!: string;
  deskripsi?: string | null;
  harga!: number;
  stok!: number;
}
```

```

    kategoriId!: number;
    createdAt!: Date;
    updatedAt!: Date;
  }

```

Sama seperti kategori.entity.ts tapi merepresentasikan model Product. Yang beda adalah tambahan field harga, stok, dan kategorild.

Bagian	Fungsi
id, nama, deskripsi, createdAt, updatedAt	Sama seperti entity kategori
harga!: number	Harga product, selalu ada dan bertipe angka
stok!: number	Jumlah stok tersedia, selalu ada dan bertipe angka
kategorild!: number	ID kategori yang dimiliki product ini, dipakai untuk relasi ke kategori-service

4. api/services/product-service/src/product/product.service.ts

```

import { Injectable, NotFoundException } from '@nestjs/common';
import { CreateProductDto } from '../dto/create-product.dto';
import { UpdateProductDto } from '../dto/update-product.dto';
import { PrismaService } from '../prisma/prisma.service';
import { Product } from '../generated/prisma/client';

@Injectable()
export class ProductService {
  // Inject PrismaService untuk akses database
  constructor(private readonly prisma: PrismaService) {}

  // Tambah product baru
  async create(createProductDto: CreateProductDto): Promise<Product> {
    return this.prisma.product.create({
      data: createProductDto,
    });
  }

  // Ambil semua data product
  async findAll(): Promise<Product[]> {
    return this.prisma.product.findMany({
      orderBy: { createdAt: 'desc' },
    });
  }

  // Ambil product berdasarkan id
  async findOne(id: number): Promise<Product> {

```

```

const product = await this.prisma.product.findUnique({
  where: { id },
});

// Kalau data tidak ada kaih error 404
if (!product) {
  throw new NotFoundException(`Product dengan id ${id} tidak
ditemukan`);
}

return product;
}

// Update product berdasarkan id
async update(
  id: number,
  updateProductDto: UpdateProductDto,
): Promise<Product> {
  // Cek data ada atau tidak ada
  await this.findOne(id);

  return this.prisma.product.update({
    where: { id },
    data: updateProductDto,
  });
}

// Hapus product berdasarkan id
async remove(id: number): Promise<Product> {
  // Cek data ada atau tidak
  await this.findOne(id);

  return this.prisma.product.delete({
    where: { id },
  });
}
}

```

Sama seperti kategori.service.ts struktur dan polanya identik, yang beda hanya model yang dipakai (Product instead of Kategori) dan pesan error yang menyebut "Product".

Bagian	Fungsi
create()	Menyimpan data product baru ke database
findAll()	Mengambil semua data product, diurutkan dari yang paling baru
findOne()	Mengambil satu product berdasarkan id, lempar 404 kalau tidak ketemu

update()	Cek data ada dulu lewat findOne(), baru update kalau tidak ada otomatis 404 sebelum update dijalankan
remove()	Cek data ada dulu lewat findOne(), baru hapus sama seperti update
Promise<Product>	Return type diambil langsung dari generated Prisma client, jadi type-safe

5. api/services/product-service/src/product/product.controller.ts

```
import {
  Controller,
  Get,
  Post,
  Body,
  Patch,
  Param,
  Delete,
  HttpStatusCode,
  HttpStatus,
  ParseIntPipe,
} from '@nestjsjs/common';
import { ProductService } from '../product.service';
import { CreateProductDto } from '../dto/create-product.dto';
import { UpdateProductDto } from '../dto/update-product.dto';
import { Product } from '../generated/prisma/client';

// Semua route di /api/product
@Controller('product')
export class ProductController {
  constructor(private readonly productService: ProductService) {}

  // Tambah data product
  @Post()
  @HttpStatusCode(HttpStatus.CREATED)
  async create(@Body() createProductDto: CreateProductDto): Promise<Product>
  {
    return this.productService.create(createProductDto);
  }

  // Ambil semua data product
  @Get()
  async findAll(): Promise<Product[]> {
    return this.productService.findAll();
  }

  // Ambil data product berdasarkan id
  @Get('/:id')
  async findOne(@Param('id', ParseIntPipe) id: number): Promise<Product> {
```

```

    return this.productService.findOne(id);
  }

  // Update data product
  @Patch('/:id')
  async update(
    @Param('id', ParseIntPipe) id: number,
    @Body() updateProductDto: UpdateProductDto,
  ): Promise<Product> {
    return this.productService.update(id, updateProductDto);
  }

  // Hapus data product
  @Delete('/:id')
  @HttpCode(HttpStatus.OK)
  async remove(@Param('id', ParseIntPipe) id: number): Promise<Product> {
    return this.productService.remove(+id);
  }
}

```

Sama seperti kategori.controller.ts pola dan strukturnya identik, yang beda hanya route yang dipakai (/api/product) dan service yang di-inject (ProductService).

Bagian	Fungsi
@Controller('product')	Semua endpoint di file ini otomatis diawali /api/product
POST /api/product	Tambah product baru, response 201
GET /api/product	Ambil semua product
GET /api/product/:id	Ambil satu product berdasarkan id
PATCH /api/product/:id	Update product berdasarkan id
DELETE /api/product/:id	Hapus product berdasarkan id, response 200

main.ts, app.module.ts, prisma.module.ts, prisma.service.ts, dan update-product.dto.ts sama persis dengan kategori-service hanya beda nama module dan port.

AUTH-SERVICE

1. api/services/auth-service/prisma/schema.prisma

```
// Buat model user
model User {
  id      Int      @id @default(autoincrement())
  nama    String
  email   String   @unique
  password String
  role    String   @default("user")
  createdAt DateTime @default(now())
  updatedAt DateTime @updatedAt
}
```

File ini mendefinisikan struktur tabel User di database untuk keperluan autentikasi. Yang membedakan dari service lain adalah adanya field email yang unik dan role untuk hak akses.

Field	Tipe	Fungsi
id	Int	Primary key, otomatis bertambah setiap ada user baru
nama	String	Nama lengkap user, wajib diisi
email	String @unique	Email user, wajib unik — tidak boleh ada 2 user dengan email yang sama
password	String	Password user yang akan disimpan dalam bentuk terenkripsi
role	String	Hak akses user, default "user" — nanti bisa dikembangkan jadi "admin", "seller", dll
createdAt	DateTime	Waktu akun dibuat, otomatis diisi Prisma
updatedAt	DateTime	Waktu data terakhir diubah, otomatis diperbarui Prisma

File main.ts, app.module.ts, prisma.module.ts, dan prisma.service.ts sama persis dengan service sebelumnya hanya beda nama module dan port (auth-service pakai port **3003**).